

ADR-0002 — DDD feature folders with hexagonal layering and an async event bus

ACCEPTED Date: 2026-09-02 · Deciders: Nicolà Widmer

Context

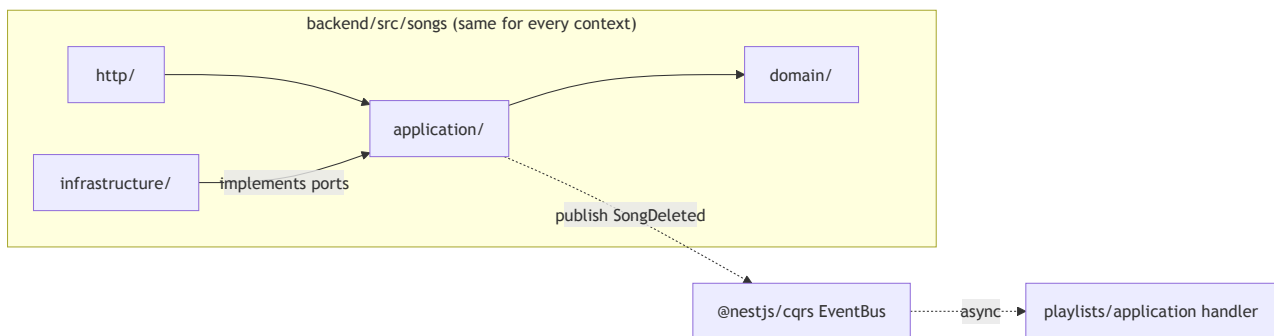
The backend holds the rules: upload checks, playlist order, ownership, and deletes that affect playlists. Use cases must be testable without a database. Deleting a song must not fail because playlist cleanup fails.

Options

1. **DDD + hexagonal**: one NestJS module per bounded context, ports and adapters, async domain events between contexts
2. MVC with an active-record ORM
3. Transaction script (one function per endpoint, inline SQL)

Decision

Option 1.



- `domain/` imports nothing from NestJS, the ORM or other contexts.
- `application/` holds use cases and port interfaces.
- `infrastructure/` holds Drizzle repositories and file storage.
- `http/` holds controllers and DTOs.
- Contexts import only `shared/` and themselves. ESLint enforces this.
- Events are fire-and-forget. An hourly sweep repairs a dropped reaction.
- NestJS provides modules, DI and the event bus. `@nestjs/swagger` generates `openapi.json` from the DTOs (ADR-0006).

Consequences

- Good: domain logic is unit-testable with fake ports.
- Good: storage and other adapters are swappable.
- Good: a failing handler cannot fail the original request.
- Bad: more indirection than MVC (ports, mapping, wiring).
- Bad: cross-context state is eventually consistent; the sweep is required.
- Bad: controllers and DTOs are NestJS classes; backend is CommonJS.
- Rejected MVC: logic on models is hard to test without the database.
- Rejected transaction script: rules get copied per endpoint.